

Customer Service System — past, present and future

P F Harrison

BT's Customer Service System (CSS) is the core operational support system for BT and brings in 70% of BT's income. It was rolled out nationally in 1989 and provided an integrated system for telephony — order handling, repair handling and billing. It continues to grow in size and scope to handle new products and services and increasingly is interfaced to other systems to facilitate automation of business processes. The need to overcome its limitations drives an evolution process for the system which is leading to the development of complementary platforms with which CSS must interwork and which will eventually replace it.

1. Introduction

BT's Customer Service System (CSS) is one of the largest and highest performing IDMS database installations in the world. It is BT's core operational support system, bringing in 70% of its income.

CSS was built to overcome a number of problems that the business faced in providing an improved customer service in the 1980s. These covered such aspects as the lack of an integrated view of customer data for telephony — orders, faults, bills and no end-to-end automation for provisioning of service. Although CSS met these objectives, its design does not meet today's requirements. The design around a geographically-based organisation means that CSS only provides a geographic, PSTN-based view of a customer. The product-based structure means that it is not possible to provide a holistic view of a customer's relationship with the company and the inflexibility of parts of CSS inhibits the company's ability to bring new products to market quickly.

The growth in CSS usage (22% of on-line use per annum) presents technical challenges as the load approaches the limits of the manufacturer's hardware. There are potential issues raised by the continued dependence on the IDMS database platform, e.g. increased shortage of specialist skills, and dependence on a single supplier in a declining market. These issues are being actively and successfully managed. A strategy exists for managing CSS operational growth for the next five years.

It is in this context that continued investment and proactive migration of CSS is required. A 'big bang' solution to replace CSS is not practical, even if it were desirable — it would take years, cost millions, and be very high risk.

The strategy is to evolve CSS on a number of fronts over a number of years while managing the current technical platforms. At the same time it will be necessary to develop and evolve other components of the operational support systems (OSS) to meet those requirements that CSS is not able to meet.

The CSS migration strategy aims to manage the scope of CSS downwards over time by migrating functions to other system components, as they appear, and to continue to improve interworking with these components. This will enable continued and improved integration across systems.

This paper presents an overview of CSS and then describes the key options for its evolution. A number of initiatives are in progress but others are still subject to further design work. The options are presented after a discussion of the business drivers which present the business context in which they must be considered.

The evolution strategy is the result of an ongoing review of the future of CSS. A considerable amount of work was undertaken during 1994 to identify the way forward for CSS [1, 2]. The evolution strategy is described in Harrison [3].

Given the scale and scope of CSS, this paper deals only with a high-level description of the system and the high-level strategic influences and consequential changes. The CSS Programme Plan [4] contains the detailed development plan; this typically records planned new functionality for the next 12–18 months. The 'Essential Guide to CSS' [5] provides a summary of current CSS functionality.

2. System overview

CSS is a key BT computer system which is vital to the health and financial survival of the company. It is currently the company's largest computing system; it manages approximately 70% of BT revenue and has been the cornerstone of BT's operations since it was introduced nationally in 1989. CSS has improved customer service while increasing the effectiveness of BT people — it is a key enabler in the management of change in BT and supports a number of programmes across the business.

CSS manages all aspects of customer service on the public switched telephone network (PSTN) from order capture, through issue and fault handling, to work management and billing. CSS deals with engineering time through national job recording (NJR), supports network management, call itemisation, line testing and many others. The CSS headquarters system allows national changes (e.g. new products) to be introduced in a uniform, speedy and cost-effective manner.

CSS supports many of the key business processes. The information held represents over 1400 gigabytes of information — equivalent to 50 miles of library shelving (the British Library contains 300 miles).

CSS manages 28 million exchange lines and each day:

- £30 million of revenue is handled,
- 433 000 bills are produced,
- 34 000 repairs are dealt with,
- 120 000 orders are received — a massive increase in the last 12 months,
- 175 000 reminders are sent out.

More detailed statistics are given in Table 1.

2.1 CSS application areas

CSS comprises numerous sub-systems which can be grouped into application areas as described below. None of these sub-systems can be run in isolation as they depend on common data-stores held across the whole of CSS.

Accounting

The accounting function involves three primary activities — processing of accounting data generated by five feeder systems, production of monthly accounting reports, and generation of the file for the general ledger accounting system.

Table 1 CSS statistics.

Statistic (size or performance)	Jan 95	Aug 95
Customers	24 million	24 million
Exchange connections	28 million	28 million
Bills produced daily	400 000	433 000
Reminders produced daily	170 000	175 000
Orders taken each day	38 500	41 000
Repairs dealt with each day	32 000	34 000
Systems — 29 separate images plus one national table build system	30	30
System size per average system	350 Gigabytes	500 Gigabytes
Database records per average system	148 million	170 million
Database records on the largest system	228 million	300 million
Database store per average system	100 Gigabytes	350 Gigabytes
Database store on the largest system	180 Gigabytes	650 Gigabytes
Database record types	> 700	902
Database inputs/outputs per transaction	8 — 12	8 — 12
Terminals per average system	4000 — 5000	4000 — 5000
Transactions per day per system	700 000 — 1000 000	700 000 — 1350 000
Transactions/day — all systems	16 500 000	23 000 000
Transmission switched transactions	—	8700 000
Seconds internal response time for home transactions	0.3 — 0.8	0.12 — 0.3
Seconds internal response time for switched transactions	—	0.35 — 0.75
Seconds response time at the terminal (1 second for SPRN networks and up to 4 seconds for TNET)	1.5 — 4	1 — 4
Lines of code	3586 013	3763 673
On-line programs	1575	1656
On-line screens	4216	4443
Batch programs	1328	1539

Customer account management

The invoicing facilities in CSS cover the whole range of activities needed to bill customers for products and services. Comprehensive up-to-date billing information is available at both the account and installation level. Customers can also be handled on a more individual basis, e.g. full details of itemised calls, monthly and temporary service billing and flexible follow-up.

Directory and operator services

The directories function provides both on-line and batch facilities for the management of customers' phone book and directory enquiry listings supporting the delivery of new phone books and operator services.

Job recording

CSS national job recording is an on-line process which provides facilities for different methods of time input for BT employees. It allows for inspection, reconciliation and on-screen authorisation of time records with immediate feedback in the event of input error.

Marketing

Marketing data is extracted from each CSS database, consolidated on a national basis and forwarded as required to marketing systems.

Materials and stock control

CSS maintains a national catalogue of stores items and provides information about stock availability, product forecasting, etc.

Network systems

The network systems support provisioning and fault management, they provide information and allocate facilities for directory and equipment numbering and the copper access network.

Order processing

The order-processing function uses an interrelated set of sub-systems to support the issuing and control of work on CSS. Job and activity types provide a common approach to dealing with both order and repair requests. CSS gives users the ability to modify job progression and automatically monitors and reports on possible failure to deliver to a customer commitment (orders in jeopardy).

Repair handling

The repair handling functionality brings advanced administration facilities to the reception and progression of PSTN and customer premises equipment faults. Integration with modern line test systems gives CSS users the ability to test lines recorded on a CSS database.

Service provision

All incoming customer contacts with BT for PSTN products and services are recorded via CSS. Customer details such as names, addresses and installations are maintained along with the ability to take and progress customer orders. Comprehensive details of all BT products and services, both national and local, are also available.

2.2 BT programmes supported

This section gives a brief summary of BT programmes supported by CSS.

Billing

CSS is involved in delivering tactical billing developments as well as strategic development, such as per second pricing and flexible billing packages for the mass market; it will interwork with BT's multi-service billing system which is designed to ensure that BT can bill its customers in the most flexible and effective fashion.

Access Network Support Systems (ANSS)

Changes to the network functions on CSS will assist in the successful implementation of the ANSS programme. ANSS proposes the introduction of new technologies into the access network supporting telephony and non-voice systems.

Customer Service Improvement Programme

The introduction of the national job recording, resource roster and appointing systems in CSS supports the company's need to respond in a more proactive way to customers' requirements. Engineers will be able to work in flexible patterns which will extend both the working day and week from the customers' viewpoint — the CSS system availability has been extended to support this. Further enhancements to roster management are being implemented.

Work Manager

Work Manager manages the work load of the engineering work force. With knowledge of an engineer's home address, office location, skills and time available an appropriate allocation can be made between the order/repair taken on CSS and the engineer who will complete that work. Engineers are now able to initiate line tests from hand-held terminals linked to CSS.

PSTN provision

Improvements in the management of orders and the service provided to customers includes automatic connection to the OMC and hence the exchange.

External regulation

Changes to VAT, deposits and competitor access, etc, which have an effect on CSS, are contained in this programme.

CUSTOMER SERVICE SYSTEM

Front-office operations and automation

This programme specifically handles customer reception systems which sit on top of CSS, providing intelligent support systems for CSS users (e.g. SMART).

Reducing fault and repair visits

The operations and maintenance centre (OMC) links and line test systems facilities identify the reasons for faults and in some cases fix them without the need for a visit to a customer's premises. It is also possible in some cases to identify and fix faults before the customer has recognised that one exists.

Cashless

This programme provides enhanced facilities for chargecard customers

Telemetry

The PSTN network is used to transfer small amounts of data. Initial use is for meter readings for utilities (electricity, etc).

Number portability

This enables a customer to retain their telephone number when they transfer from one service to another. This is an OFTEL requirement.

Engineering effectiveness with quality

This programme is designed to improve customer service by enabling engineers to take action if they can see potential problems arising. This programme will actively encourage the engineer to carry out work to save a fault developing in the near future. Details will be entered via CSS and forwarded to the relevant management information system.

OMC

This programme supports the management of digital exchanges for all remote operations and maintenance.

Direct fulfilment

Customers are provided with a next-day service for the supply of a plug-in telephone.

2.3 The CSS database

The CSS database reflects the many complex relationships between the various components and

processes which constitute the provision of customer service. The database is managed by the Integrated Database Management System (IDMS).

The complex relationships between real-world items (customers, lines, exchanges, bills) and processes (bill production, order handling, repair handling) are represented in the IDMS database schema. In addition, IDMS provides control for:

- concurrent access requirements,
- update tracking and monitoring (transaction logging),
- file management,
- full recovery procedures which can be invoked if any type of failure occurs.

Performance features include the following.

- Read-only service (this service is only available on the production machines) — the contention for access to database records, resulting in long wait times for CSS users, has been alleviated by the provision of a separate facility (the 'read-only CV' (central version)) for CSS interrogations. Transactions which need to read but not update the database do not require as much management as those which do need to update (e.g. there is no need for recovery records to be maintained). The top 20 transactions used on CSS are read-only and savings have been made by the provision of the facility to separate out different types of access requirements.
- In-flight dumps — in order to ensure that data can be recovered in the event of failure, frequent database dumps are required. In the early days of CSS, the system was taken down for 2–4 hours while these dumps were taken. The current business requirements are for 7-day, 24-hour working and a process for 'in-flight' dumps of the database is now available to allow the computer centres to take secure copies of the system while end users continue to have access.
- DC-shipping — the CSS transaction processing management system is IBM's CICS. IDMS itself has a transaction processing monitor which is known as IDMS-DC. To take advantage of the two products, and to make better use of machine capacity, a facility has been provided which allows the centres to dynamically shift processing from IDMS to CICS and vice versa. Some CSS transactions are developed using both CICS and IDMS-DC access commands and both versions are shipped to the computer centres.

2.4 Interfaces

The system was originally designed to be largely self-contained. However, while much functionality lies within the system, increased automation along with various company reorganisations and the ever-growing demand for management information has resulted in the number of external interfaces increasing to well over one hundred, including:

- Operations and Maintenance Centre (OMC) system,
- Work Manager (WM),
- Service Domain Processor (SDP) for elapsed time charging (ETC),
- Number Information Services/Operator Services Information System,
- financial information system,
- marketing systems (IMS/ICIP/MARK),
- management information systems,
- line test systems,
- Bankers Automated Clearing System (BACS),
- SMART — front-end customer-handling system currently rolling out in the 150 channel,
- Customer Handling Intermediate Server (CHIS).

The method of data transfer can be anything from interactive messaging, electronic file transfer to a floppy disk.

2.5 CSS releases

The current release strategy shows major CSS releases taking place at two/three-monthly intervals with interim releases where urgent operational need is identified. The schedule starts to be planned at least one year ahead with major functionality identified for specific releases.

2.6 CSS Operations

CSS is designed for operation on the IBM 3090 (or compatible) series of central processors, running under the MVS operating system. The processors currently include models from IBM, Amdahl and Hitachi. Several makes of disks are in use at the various sites; these include Amdahl, EMC, IBM, Comparex and Iceberg.

Originally CSS was deployed and configured individually for each of BT's 29 geographically separate

districts. The system has undergone consolidation over recent years and is now deployed at five separate physical locations in the UK (although the system still exists as 29 images with separate databases). Operations are controlled from two command centres and three main print centres are used. Transaction switching allows users on one CSS system to access and interact with data held on another CSS system. This significantly enhances the seamless nature of the national CSS system and national processes are introduced and managed centrally, while also giving local flexibility. There is a 30th CSS system which is used by the national table builders to build, update and distribute national data tables.

The configuration of CSS at each computer centre is maintained by BT's Computing Services and Operations (CSO). They provide support for users, applications, operations, network and technical issues, and operational database administration functions. CSO act as an interface between operational managers, users and the development groups within the systems engineering centres.

The connection of the user workstations (IBM 3270-compatible terminals and remote printers) to the system is via an extensive communications network based on both SNA and TNET network protocols. Up to 60 000 terminals have access to CSS.

2.7 CSS software environment

The software environment is achieved by combining standard IBM software, specifically developed BT common software (i.e. middleware), and the application software developed for each sub-system. The software environment is detailed in Table 2.

Table 2 The CSS software environment.

Operating software	MVS common system
Database management system	CA/IDMS
Data dictionary	IDD
Programing language	COBOL II, ASSEMBLER, EASYTRIEVE, 370 'C'
Screen editor	SDFII
Job scheduler	CA7
Problem reporting/management	Infoman
Software control/configuration management	BT XMODS
Tape management	CA1
Job entry system	JES
Job control languager	JCL
Command centre controller	AOC automated operator control

2.8 Common middleware

Middleware is a set of reusable BT software which was initially developed to support CSS but is now used by other programmes (e.g. COSMOSS) and projects [6]. Middleware provides a standard interface to all areas of CSS for common tasks such as:

- sign-on,
- screen handling,
- interconnection between systems,
- application services.

It provides consistency between CSS functions and reduces processing by validating on-line input before it reaches the application-specific software. The major components of BT Middleware are Dialogue Manager, Middleware Message-Based Interface, General Messaging Interface and Transaction Switching; these are briefly described in the following paragraphs.

- Dialogue Manager performs the following functions:
 - access security vetting,
 - input data validation,
 - menu construction,
 - transaction flow control,
 - systems messaging,
 - control of the function keys (commonly referred to as the PF keys),
 - systems interface, performing a common means of connection to other systems over communication lines in real time, i.e. line test, Work Manager and equifax,
 - data level security.
- The Middleware Message-Based Interface (MMBI) — the advent of client front-end customer-handling systems, such as SMART, means that these systems need to communicate with CSS in a way which will provide fast response times and will not place undue overheads on the CSS machines. MMBI has been developed to give this flexible, responsive interface. It provides the front-end systems with a degree of independence by de-coupling CSS data and function from CSS screen formats using a self-defining message protocol over the existing 3270 (terminal) interface. The client can be protected from CSS screen changes while gaining access to existing CSS functionality.

MMBI also provides the ability to execute 'objects' on the CSS host, each comprising a sequence of standard business transactions. These objects are in the form of a 'script' held on the host containing the logic to drive existing transactions in the required sequence. A simple script language has been developed which provides flexibility in execution, i.e. a script is able to react to the value of data items, messages, etc, returned on a screen and determine the next course of action. The MMBI approach provides performance benefits for front-end clients over 'screen-scraping' as, typically, only a single network interaction with the host is required to execute a sequence of transactions.

- General Messaging Interface (GMI) — the GMI is a further development, to provide client/server-style access to CSS. It is now possible to pass messages into middleware (and hence CSS) in the same manner as for MMBI, but via other communication protocols (e.g. Logical Unit 6.2 (LU6.2) and Transmission Control Protocol/Internet Protocol (TCP/IP)). This also enables non-screen-based transactions to be developed for client/server interactions.
- Transaction Switching — the transaction switching development was undertaken to realise the BT objective of a single point of contact for all customer enquiries concerning PSTN service. This initially supported the front-office project, but has been extended to support other business units using CSS.

Transaction Switching enables a user to perform tasks on one of a number of linked CSS databases with only one sign-on being required. The system determines, from data entered by the user, the CSS database on which the requested transaction will be run. The facility is transparent to users who initially sign-on to their 'home' CSS system. The process relies on the ability of the dialogue manager to inspect the user input and decide whether to invoke the transaction at the 'home' system or to route the request to an 'away' system in the transaction switching network.

2.9 Robotics and CSS

The use of PC robots has increased substantially over the last few years. PCs with terminal emulation software are used to automate large-volume repetitive operations such as order entry for discount options, direct debit, chargecards, etc. The robotic software mimics the role of a CSS user and, supplied with input data, will execute a predefined set of CSS transactions. From a single PC, CSS changes can be applied not only to the local CSS database but, via transaction switching, nationally. The main application areas are in billing, and marketing and sales.

Robots can be highly successful when used in harmony with careful scheduling. However, their use can affect the overall performance of CSS and are therefore implemented with care.

3. The business drivers for evolution

The key business drivers will continue to be a balance between the following requirements:

- to provide a comprehensive customer-support service across all products and services to maintain market share with a focus on comprehensive personal communications and total business solutions,
- to increase the effectiveness of the sales force and business by improved targeting and selling, with a particular focus on the most valuable segments and increasing, for example, global and interconnect markets,
- to simplify and improve the business processes to improve cost effectiveness and empower individuals,
- to establish internal trading units bringing a greater focus on profitability and hence the need to reduce costs, as well as the need for systems to support the independent operation of trading units,
- to exploit new technologies and develop new products and services and in particular speed up the development of advanced services and time to market.

Currently, the main challenges facing the CSS programme include the increasing level of use — more users over a longer BT working day and week (25% rise in 95/96) — and the increasing volume of change — CSS is a key provider of new and enhanced functionality for BT (30% rise in 94/95 sustained in 95/96). Hence some detailed objectives are:

- simultaneous on-line and batch operation to support the lengthened BT service day,
- improving the operational performance, in particular fast response times, and reducing operational downtime — availability must be as close to 100% as possible,
- responding to requirements in a more flexible fashion,
- managing competing business priorities,
- end date driven developments — increasingly enhancements and new developments have to be in place to meet specific business and OFTEL target dates,
- the changing role from a prime service provider to users to a server system supporting the new front-end systems,

- added complexity of delivering interlocked business developments focused on CSS;

and CSS provides BT with:

- single point of contact — integrated channel management,
- unified database — a single database used by a number of sub-systems,
- customer details (for instance) will always be the same across all the functions,
- cost reductions associated with improved processes and increased automation,
- the management of 28 million exchange lines and the associated bills, orders, etc.

CSS has therefore overcome the limitations of the systems which existed prior to its rollout when there were separate systems to process bills, orders and faults. It was impossible to gain a single view of a customer's installation, with the data in the multiple systems often being inconsistent; there was limited on-line data, there were extended provisioning times, and there was no integrated management information.

In the present competitive environment, there are now requirements which reveal the limitations of the CSS design. CSS can only provide a PSTN view of a customer and even this is constrained by the geographic divisions in the system. Current requirements are that systems must provide a holistic view of a customer's relationship with the company — across product and geographic boundaries. Some of these limitations include:

- speed of change — new requirements take a long time to go through the development process from requirements to deployment,
- flexibility — it is often necessary to make software changes instead of reconfiguration to deal with product changes,
- CSS was designed to manage service over copper in the access network — technologies such as optical fibre cannot easily be managed on CSS,
- extensive user training is often required, the business now wants customer-facing people to focus on dealing with the customer and not driving the system — processes must therefore be simpler and be driven by systems.

The required solutions will increasingly be delivered by systems other than CSS. Thus the future for CSS and indeed

all operational support systems is one of increased interworking with other systems to maintain and provide seamless end-to-end service, particularly during the migration of functionality from one system to another. CSS will also be subject to continued enhancement to support process improvements and new products and services that lie within the scope of the system, e.g. volume billing products and basic PSTN products.

Since the introduction of CSS there have been further developments in communications technology — fibre, intelligent networks, etc — which will enable the provision of more complex and flexible services that will require adaptability and support beyond that for which CSS is designed. Competitive edge is not just a question of the excellence of the technology provided to the customer but increasingly it is about the quality of the service particularly at the customer's point of contact with the business. Although there is ongoing development of new functionality on other systems to support this requirement in such areas as customer handling and billing — SMART, MCSS, MSBS — nevertheless, CSS will still remain an essential element in the provision of service excellence for PSTN and volume products and services for the next five to ten years.

Many requirements are now channelled through major programmes which, in defining their strategy, define the space in which CSS operates and hence enable the definition of the future for CSS. CSS evolution is driven by and must align with these programmes, but must also feed into them to help shape their strategies. The following sections review the current major programmes and their impact on CSS. However, it is important to recognise that the business objectives will change and hence the impact on systems will change. It is necessary to constantly review the evolution strategy and adjust it as the underlying assumptions are themselves adjusted.

3.1 *Customer handling*

The development of front-end customer-handling systems, such as SMART [7], ACE [8], MCSS [9], etc, requires a change in the role of CSS to that of a back-end server in a client/server architecture. Significant progress has been made in understanding the impacts and in initiating changes. Client/server developments in the short term increase load on CSS but have the potential to reduce the load via long-term migration of function away from CSS and hence provide an evolution path. The development of the Operational Customer Database to meet customer-handling requirements also provides the mechanism to overcome some of the limitations in the CSS customer data structures.

3.2 *Billing*

The first release of the new multi-service billing system will be delivered during 1996/97 which will provide billing function for a range of customers and products. Over the next three to four years the billing programme will focus on the provision of services to all business customers and subsequently the top 20% of residential customers. CSS will remain the key billing system for the next five years for both volume business and residential. As there are currently no plans to migrate 'simple' residential customers, the CSS billing function will continue to need to be enhanced to support new products and services.

3.3 *Provisioning programme*

The provisioning programme is focusing on improvements in the end-to-end service and on support for new products and services, particularly complex products and emerging technologies. As a general-purpose order handler and job manager, possibly based on COSMOSS, will be evolved and will be used increasingly for new products and services, interworking between CSS and other provisioning platforms will be essential (both directly and/or via new mid-tier provisioning applications). This, together with customer handling, is now the key area for reducing the load on CSS and the migration of functionality.

3.4 *Access network support*

The ANSS is concerned with carrying out the provisioning and repair of services over new access technologies. CSS will continue to be the repository for copper records including enhanced copper technologies. ANTAS will carry out assignments over enhanced copper and interwork with CSS.

3.5 *Problem management*

CSS will continue to be the fault-handling system for PSTN for the next two years. The major change is that CSS will interface to the generic test controller (GTC) which will protect CSS from changes in this area. It is further expected that the mid-tier will interface directly to the GTC so that the front-ends will be able to initiate line testing directly and bypass CSS.

There are also proposals that a mid-tier problem diagnosis and workflow capability be developed which will be able to manage problems across a wider range of services than the current product-specific systems.

4. Technical drivers for evolution

There are a number of current issues ranging from increasing demand for the system, which puts a strain on the hardware capacity, through to architectural constraints, which inhibit the ability to respond to business requirements. These and other issues are considered here.

4.1 Growth in demand

Current trends show demand for on-line use growing at 22% per year while batch growth is 10%, the central role of CSS means that complete lack of availability or poor performance can cause significant disruption to the business. A key constraint is that all database update activity must be performed in a single processor using the current version of the IDMS database software, the next release of IDMS (Release 14) will solve this problem. Currently, increased load in other parts of the system can be spread across multiple processors. Many of the current processors do not have sufficient capacity for the expected growth in database activity on the largest databases. Hitachi is the only supplier with a single processor engine capable of dealing with the largest load CSS will place on IDMS and a number of these have been deployed during 1996.

The growth figures are based on the trends achieved over recent years and the assumption that the pattern of usage remains much as now. The actual load and demand for every CSS image is monitored every month. The most significant risk is uncertainty surrounding the demand figures. The recent spring sale doubled processing in some areas of CSS and this has been accommodated.

4.2 Hardware evolution

Currently available 'Summit'-class mainframes, based on ECL (emitter coupled logic) technology, have processor sizes of approximately 60 RPP (relative processor power). Hitachi Skyline processors are now being used for the largest CSS images. The current estimate of the Skyline processor capability on a CSS is approximately 110 RPP, this is not expected to increase substantially.

At current growth rates the IDMS update load on the two largest databases will exceed 110 RPP by the year 2000; however, two ameliorating factors need to be considered — the changing capability of the system software, primarily CICS and IDMS, to be able to manage multiple processors, and the evolving hardware technology.

Hitachi are the only supplier to have chosen to develop ECL processors of this size (110 RPP); both IBM and Amdahl are concentrating their efforts on CMOS technology and the provision of multi-processor

architectures. Currently installed IBM CMOS processors are rated at about 25 RPP. Amdahl are forecasting that their Millennium CMOS engines, available from mid-1996, will provide 40—50 RPP. Industry forecasts are that the power available from CMOS processors will double every 2 years or so, suggesting that by the end of the century, CMOS processors will be more powerful (and cheaper) than ECL based ones. Amdahl have confirmed that their plans align with these projections.

The average cost per RPP of mainframe processors has fallen 80% between 1990/91 and 1995/96 and is projected to fall 75% between 1995/6 and 2001/2. Storage costs are also falling by similar proportions.

The situation is a dynamic one and the choice of hardware will be continually reassessed on the basis of cost, capacity and in the light of the capabilities of system software to exploit multi-processor architectures.

4.3 System software constraints

The Monte Carlo project, initiated to monitor and manage growth in processor load, has introduced a number of very effective methods of managing the load which extends the life of the current processors — namely splitting the on-line transaction load across a number of CICS systems and splitting the database load across a number of IDMS engines. The main concern is that for IDMS the update activities must all run in a single processor, and on this basis CSS growth will exceed the capability of a single processor. Off-loading retrieval activity to separate read-only database processors has reduced the problem and it has only been necessary to migrate to Skylines for the largest CSS images where the update activity exceeds the 60 RPP limit of the Summit-class processors. Further work is planned that will provide a 20—30% saving in transaction load on the database update processor. This performance balancing activity makes effective use of existing hardware, but, assuming current growth rates continue, the ceiling of the Skyline processors will be reached within five years. The introduction of IDMS Release 14 which supports multiprocessor working will relieve this problem.

This strategy will enable the CSS operational load to be managed for the next five years if IDMS 14 is implemented within three years and assuming current rates of growth.

4.4 Future of IDMS

Although currently CA are continuing to evolve the product, e.g. to keep pace with hardware advances, there is concern that IDMS is an ageing product. Also there is increasingly a shortage of specialist IDMS skills which are also growing in cost.

Recent reports and attendance at IDMS user conferences indicate that IDMS still has a significant user base and that this will continue for some time and that CA are continuing to invest in the product.

However, in the longer term BT's strategy is to reduce the system in size and scope so that the remaining parts can be ported to a new platform, if appropriate (see section 6).

Another major concern is the dependence on IDMS skills. BT is currently one of the largest employers of IT people with IDMS skills. There is an increasing reluctance among people to work on IDMS-based systems due in part to a general perception that the future lies with open systems. Many of BT's systems are on mainframes and are likely to remain so for many years. Increasingly IT people will need skills across a range of hardware and software environments, including IDMS and IBM mainframes open systems and PCs.

4.5 Workstack bottle-neck

CSS is a large complex system which forces too many requirements through the same bottle-neck — this is particularly apparent in billing and order handling. The current workstack is very large but comparable to that of other systems of its size. Additionally, with the growth in new products and services to enable BT to remain competitive and gain new business, the workstack is growing faster than developments can be undertaken and hence there is an increasing backlog of work.

In the short term, the programme has developed a two-year plan which enables the development resource to be managed to meet demand more closely. The issue then is how much is it right to spend on CSS development each year. The past growth has been 25% year on year; development investment is now expected to continue at the present level.

In the longer term, the CSS workstack bottle-neck can only be removed by spreading the work across a larger number of systems so that work can proceed in parallel. This means the development of alternative and complementary platforms which can meet new requirements and to which function can be migrated. This, of course, will bring new challenges of integration, testing, complex delivery and performance — and the bottle-neck may only have moved somewhere else.

4.6 Architectural constraints

The overall design of CSS constrains BT's ability to use it to deploy new business functionality, for example:

- lack of flexibility — while some parts of CSS were designed to be flexible and are data driven, there are limitations in this which often require new software development to support new products and services where it ought to be possible via data configuration,
- new requirements for which the platform was not designed — a key constraint imposed by CSS is in the provision of a business view of customer data, i.e. it is not possible to provide a view of all of a customer's data (inventory, order, faults, contacts and accounts) across all products, geographical locations and organisational units,
- although the deployment of front-end systems removes the limitations of dumb terminals there is still a problem in that CSS transactions were not designed for client/server working.

A number of approaches can be adopted to meet these constraints. There may be scope for improving flexibility, in some cases via re-engineering existing function. In general, such requirements, for example, a business view of customer data, can only be met by the development of new platforms, in this case the Operational Customer Database, and the seamless interworking of CSS with those new platforms. Over time the existing functionality on CSS can be replaced by the new platform. The migration strategies for these issues are dealt with in section 6.

5. The way forward

The evolution of CSS consists of the pursuit of a number of activities in parallel. The immediate concerns of the capacity and technology of CSS are being actively and successfully managed as described in the preceding section.

In the long term, CSS must inevitably be replaced. However, the 'big bang' replacement of CSS is not practicable. Having built one of the largest systems in the world it would not be possible to develop a replacement set of systems and to cut over to them in any sort of 'big bang' manner. The cost would be enormous, it would take a very long time and would be very high risk. No other company is following this strategy, and no-one can afford to stand still and wait while their systems are replaced.

The approach must be evolutionary, migrating functions to other platforms over a period of time such that each migration step meets new business requirements. New platforms will be developed to meet new requirements and it is the development of these platforms which will enable the migration of CSS. In time, CSS will be reduced in scope until the size is such that the replacement or modernisation of the remainder will be a feasible task on its own.

It will, however, be necessary to continue enhancing CSS to meet business requirements until there are alternative platforms to deliver the required functionality.

There are a number of possible strategies for forward migration of CSS to meet the needs of the business. It must be remembered that CSS consists of 60 sub-systems covering a broad range of functions which are interconnected via an integrated database. The options for migrating CSS will be different for each part, requirements will differ and adopting a diversity of approaches also spreads risk across a number of technologies and suppliers.

The following approaches are being pursued:

- making improvements to CSS via addition of new functionality and the re-engineering of existing code to improve maintainability and flexibility,
- porting code to new database platforms,
- rationalisation of functionality by moving function to alternative platforms,
- development of new functionality to supplement CSS,
- improved interworking with other platforms including client/server interworking,
- investigation of use of migration technologies with external suppliers.

The activities described in section 6 show the application of these approaches. No one example will necessarily employ a single approach. Many of these activities are already under way, some being initiated as a result of the 1994 design options work [2].

6. Strategies for migration

This section will describe the major CSS migration activities. Many of the activities described here are concerned with integrating CSS into new platforms on which new functionality is being deployed. The key limitations of CSS — holistic view of customer data and inflexibility — are being addressed in these new developments. The new components are the operational customer database (OCDB), mid-tier order-handling and problem management; and a number of front-end systems. It is paramount that these components interwork to ensure that a seamless service continues to be supported, the customer-handling intermediate server (CHIS) provides the 'glue' which will enable this.

Re-engineering for interworking is not the only strategy, however; decoupling function and migrating to new platforms is also being pursued, as is the re-engineering of parts of the system to prolong their life and introduce flexibility and there will continue to be a search for success

stories (and failures) from across industry and the academic world.

6.1 Approaches to re-engineering

There is a wealth of material on re-engineering and reverse engineering, much of it focused on re-engineering code at the module level. This work has been driven by the need to make legacy code more maintainable and to seek the holy grail of identifying reusable components.

Attempts to identify and extract reusable components within legacy systems has proved to be very difficult. Recent work carried out by Durham University in collaboration with BT [10] demonstrates this very clearly; the analysis of code structure and data usage within a large CSS program shows the program to be very complex with a large degree of interconnectivity between both the 221 program sections and the 4000 data items. To continue the analysis to attempt to identify reusable components requires the availability of extensive domain knowledge. It is clear that to carry this out on an industrial scale would require considerable investment.

A second approach identifies the need for a reusable component, builds it and then modifies existing programs to call the reusable module as required. This approach was adopted in the development of a VAT module for CSS [11, 12]. The existing VAT processing is scattered among many programs and is difficult to change. A VAT module was developed using object-oriented techniques; programs are now being changed to use this module instead of performing their own VAT calculations.

Attempts to re-engineer to improve maintainability often flounder due to the imperative to deliver new function. Some work was carried out in this area in 1994 [13, 14]. The work focused on programs that had already been identified as troublesome via the volume of fault reports; complexity metrics were used which identified complex areas of the code, and tools were used to improve it. While some code improvements were taken forward into production, the general adoption of the approach would have required taking resource with expert application knowledge out of the mainstream development process and reskilling with expertise in a re-engineering toolset.

The current approach to improving the productivity and effectiveness of the maintenance task involves encouraging the use of program-understanding tools and code-measurement tools [15].

Such 're-engineering in the small' will continue to be a useful activity to prolong the life of existing components and to improve productivity for carrying out maintenance and enhancements. The fundamental problems as described

in previous sections will not, in general, be addressed by such activity. In order to provide the step change in system capability that is required, the architecture of the whole set of operational support systems needs to be considered and then re-engineering CSS can be focused on CSS's role in that architecture.

From this perspective a reuse strategy emerges — make use of existing systems for new products and services wherever possible and enhance where necessary rather than building point solutions for each new requirement. The key requirement for such an OSS is that existing systems interwork to provide a seamless service to the end user. Such interworking enables new components to be built to fill the gaps in existing functionality. In building such an architecture it becomes possible to imagine evolution scenarios for legacy systems. The emphasis moves from 're-engineering in the small' — code level components — to 're-engineering in the large' — system and architectural level components in which the OSS is seen as a single system [16].

This ties in with the strategy proposed by Brodie and Stonebraker [17]. This consists of an incremental, low-risk approach to legacy system migration. Preservation and conversion of the data is seen as the key and, in migrating the data from the legacy system to the replacement, it will be necessary to deploy gateways between old and new to broker access to data and/or function. Interestingly, they also argue that in carrying out the migration there is little value in carrying over the old code to the new system as it will embody old requirements, will be written in an archaic third-generation language, and will have deteriorated in quality over the years so as to be almost unmaintainable — migrate the data but rewrite the function in a higher-level language.

A crude but useful decision tool can be based on the diagram in Fig 1 [18]. The two dimensions are functional quality and technical quality. An application's functional quality is essentially a measure of its ability to meet user requirements. Technical quality is concerned with quality of code and documentation, cost of ownership and sustainability of the platform. The matrix provides a useful guide on the type of re-engineering to propose for any particular sub-system.

The approaches described in the next sections focus on 're-engineering in the large' evolving CSS to interwork with other components in the OSS some of which will be new and will provide complementary functionality and will provide the long-term means to supplant parts of CSS. However, there will also be cases where an existing sub-system still meets business requirements and a case can be

made for moving it to a more modern platform and at the same time overhauling the code to improve maintainability.

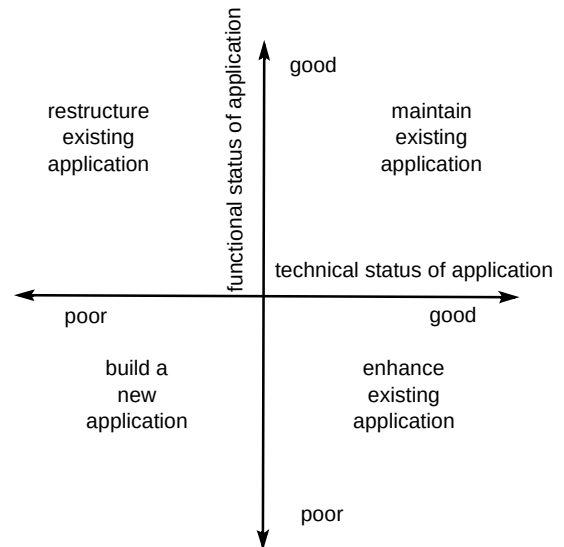


Fig 1 A re-engineering decision tool.

6.2 Client/server interworking

The development of SMART, MCSS, ACE, Versatility and other front-end systems have had major impact on CSS. Typically, as their transaction rate is considerably greater than an equivalent dumb-terminal user, performance requires on-going management. Additionally, CSS transactions were not designed for client/server working, some require modification and new functionality is required, particularly to reduce performance impact.

The first deployments of client/server systems used existing CSS transactions using 'screen-scraping' technology, i.e. the client system used terminal emulation to run existing transactions and so looked like a normal user to CSS. Such client systems are vulnerable to CSS changes, are using both an inappropriate interconnection technology and CSS transactions for applications for which they were not designed, and hence inflict an unacceptable performance overhead on CSS.

Two re-engineering activities are addressing these issues:

- middleware components have been developed to enable the efficient execution of existing transactions by client systems and the deployment of new application services for client systems,
- bespoke application services have been developed for client systems providing efficient data access and hence mitigating the performance impact of using existing transactions.

To date these middleware components and new services have been deployed for Epsilon, SMART and ACE. The approach has been proved technically and further work is under development which will reduce performance impact on CSS and hence lower operational costs.

The current approach to front ends and the client/server domain is now focused on the development of mid-tier application servers such as CHIS [19]. CHIS provides a gateway into back-end systems via a set of reusable services and a platform on to which new function can be deployed which complements that on the back-end systems. It will enable the co-ordination of, for example, complex order entry across multiple back-end systems and enable the migration of CSS function to new platforms while still providing the integrated view to the user. Varying demands by front ends on CSS and the increase in usage of CSS created by front-end facilities has increased the load on CSS; the re-engineering activity to support SMART has been particularly effective in mitigating this impact.

For successful client/server working, standard interfaces to back-end systems via common mid-tier applications are required. This will maximise reuse for front-end applications (including PC robots) and minimise consequential development on the back-end system.

A greater drive on the client/server domain would also enable migration of other functionality/interfaces out of CSS, for example, line test which should be available and initiated from the mid-tier rather than from CSS.

Additionally, further rationalisation is required to deliver data to front ends in the areas of order handling, fault handling and event notification. In the last case a client system has to interrogate CSS to examine the state of order, faults or customer contacts. The requirement is that when an order, fault or issue needs escalating CSS proactively sends the information to the appropriate customer-handling system. This approach will minimise the load on CSS and provide a common approach for all front ends.

6.3 *Interworking for provisioning*

The use of a mid-tier application server to co-ordinate placing of complex orders across back-end systems is only the first enabler for flexibility. The second is that the back-end provisioning systems (CSS, COSMOSS and other systems) need to be integrated so that they can work together, at both the order-handling and job-management levels, and together be accessed via the mid-tier, in the end-to-end provisioning of complex packages and bundles of products. Hence the flexibility of COSMOSS could be exploited at the order capture stage for new products while the basic PSTN components of those products would still be provided by CSS and the overall order is job managed by

CSS job management and COSMOSS generic job manager working in concert [20].

An example of this could be the provision of Featureline which consists of the provision of centrex facilities to a customer who may have sites which are located in areas covered by different CSS databases. CSS would provide the required PSTN lines while the number plan and overall co-ordination and jeopardy management of the order could be co-ordinated by COSMOSS.

A second example of an existing product which is poorly supported by the existing systems is ISDN30 which requires considerable manual activity to co-ordinate CSS and COSMOSS. This could be automated under the proposed re-engineering.

6.4 *Interworking with the Operational Customer Database (OCDB)*

The third key migration step is the development of the OCDB. This is currently driven by the need for customer data to support customer handling.

The OCDB will have the ability to represent complex customer structures such as organisational hierarchies and roles. Hence, this will provide the cross-reference facility that will supply a view of a customer across multiple CSS images, COSMOSS, CAMSS so overcoming the current geographic and product-specific views imposed by the current systems.

Further, the OCDB would store customer contact and event data and high-level views of customer inventory data, and later, as the convergence of CSS and COSMOSS proceeds to provide a unified order handler, the OCDB would be the master source of inventory data for products provided over the two systems.

Initially, the OCDB would be the master source of data that is not currently held in the back-end systems, such as customer hierarchies, and hold copies of inventory, fault and contact data; in time some of this data would be held as master data on the OCDB. The back-end systems would only need the data required to do their 'engine room' job and would not be concerned with, for example, customer contact data.

6.5 *Interworking for end-to-end problem management*

The current problem-management systems are tied to the existing core product set (CSS repair handling and line test for PSTN, CAMSS for private circuits) and provide little automated diagnostic capability which ensures that the problem is sent to the correct place for resolution.

Under the problem-management programme new automated diagnostic tools are proposed to be deployed at the mid-tier and, via the use of workflow tools, problems could then be routed to the appropriate place. This will provide a problem management capability that will be able to respond to new products and services which do not fit into existing product categories. To support the diagnostic tools the mid-tier will need direct access to line-test systems and therefore this function will be migrated from CSS [21].

6.6 *Re-engineering for systems interface flexibility*

As the interconnection of the operational systems increases, the deployment of individual systems becomes ever more difficult as they need to be kept in step with many other systems to work correctly. This is a growing problem for CSS as it becomes the central part of a number of automated processes connected together electronically, for example, the links to OMC and to Work Manager. A substantial piece of re-engineering needs to take place to overhaul the interfaces so that they are resilient to changes at the other end of the interface and so that they can provide multiple versions of the interface to other systems. This ability to support multiple software versions will enable components of the OSS to be deployed independently of other components as long as backward and forward compatibility is provided; a feasibility study into this is planned for the near future.

6.7 *Enhance and re-engineer — billing*

Over the next five years the multi-service billing system (MSBS) will deliver a billing capability for business and top residential customers which will reduce the load on CSS as the billing data for these customers is migrated to the new platform. CSS will be re-engineered to interwork with MSBS.

For the mass market, CSS Billing will be enhanced over the next two to three years to provide flexible discounting schemes and product packaging. As part of this the billing code will be overhauled to improve maintainability and hence extend the life of the code and to make it easier to deliver further changes. In addition, there is a proposal to re-engineer the call-processing sub-systems of CSS billing to greatly improve efficiency, maintainability and usability. This will provide an opportunity to start migration to relational technology such as DB2 or Oracle.

6.8 *Migration of function*

This section deals with possible scenarios for migrating the current CSS application to an alternative database.

The question of moving CSS to DB2 was considered in 1993 in consultation with IBM. This report proposed that the code conversion would take a year and that for the last four months of that time no new functionality could be delivered. The resulting system would have a new underlying database but would exhibit the same characteristics as the IDMS system. The IBM proposal was a 'big bang' approach which is high risk and would have prevented new functionality being delivered for a period, and was therefore considered not to be a viable approach.

There is currently an activity to clone the CSS receipting and debt management sub-system to enable its use to support MSBS. This project will take a copy of the CSS code and convert it to use DB2. In the longer term, consideration will be given to interworking between the new system and CSS and the migration of the existing CSS data.

A practical approach would be incremental in which each step is sufficiently small to minimise risk. In this way function gradually progresses to the new platform along with the people who support the code. The migration can be carried out over a longer period of time (say, 5 years) to minimise the impact on development of new function.

There are a number of key issues, four of which are detailed below.

- The design options work [2] addressed the issue of decoupling CSS sub-systems. A number of sub-systems were analysed to find out how much data was shared between them, with the conclusion that the system's components were highly interconnected. This has serious consequences for an incremental migration strategy. In migrating a subset of the database and some associated programs it will be inevitable that those programs will also need to access data that has not yet been migrated and that programs that have not been migrated will need to access both non-migrated and migrated data. The solution to this is to deploy database gateways which mediate between the old and new databases. XCall, as used by COSMOSS [22], would provide this capability, but extensive code re-engineering would be required to use it. More work needs to be carried out to understand the requirements for such gateways, their availability in the marketplace, and their performance characteristics.
- It is a well-recognised phenomenon that over time programs deteriorate in structure and quality, becoming increasingly difficult to maintain. If system migration was to consist of simply converting the database navigation this would still leave the maintainability problem.

- Performance penalties would be incurred as the system moved from its current highly tuned environment to a new database platform.
- In an approach that only aims to migrate the database platform, the existing design constraints would remain.

While database migration does not appear to solve a number of the major issues facing CSS evolution it will probably be a useful migration step for some parts of the system. There are currently two initiatives investigating this option, brief summaries of which follow.

- **RADAR**

This initiative is initially concerned with extracting CSS receipting and debt management functionality from CSS so that the code can be reused for MSBS. The project end-game envisages moving the existing receipting and debt management function from CSS to the RADAR platform.

- **National job recording**

There is a requirement that this sub-system be separated from CSS so that requirements can be decoupled from the CSS workstack. To date it has been difficult to produce a business case to justify setting up a separate system. Clearly, to migrate this functionality to another platform helps in the overall CSS migration. Another approach which needs to be considered is migrating the function to another existing platform. As Work Manager rolls out nationally it might become the natural place to carry out the NJR functions.

6.9 Network management

As provisioning functionality is enhanced on other platforms to handle orders for switched services it will become desirable to open up the CSS network-management capability for access by other systems. This is possible via the client/server mechanisms used for customer handling. While copper continues to be the main technology in the access network, there is a strong case for re-engineering the network-management sub-systems to extend their life; options here vary from re-engineering to improve maintainability and flexibility, through to porting or re-developing on a new platform.

A particular candidate for migration is number management. This resource-management function is one that is required for the provision of all switched services and should be readily accessible as a common function. Further drivers for re-engineering this function are number portability and personal numbering which are currently being delivered by CSS; this is a sub-optimal solution. If number management is likely to be a key business product and function in the future, then an alternative system for managing numbers needs to be considered.

6.10 Legacy migration tools and techniques

A number of suppliers are producing tools and techniques to enable legacy system migration. Discussions are well advanced with suppliers who can help with database migration in the RADAR project. Other technology opportunities will be pursued as they become available — currently Amdahl are offering a migration environment called ObjectStar, and Texas Instruments are offering 'Transition Re-engineering'. The most promising of such technologies will be investigated.

7. Conclusions

It is clear that continued investment and proactive migration of CSS is required. A 'big bang' solution to replace CSS is not practical, even if desirable — it would take years, involve excessive cost and be very high risk.

The strategy adopted in 1994 remains valid — evolve CSS on a number of fronts over a number of years while managing the current technical platforms. At the same time it will be necessary to develop and evolve other components of the operational support systems to meet those requirements that CSS is not able to meet. The CSS migration strategy aims to manage the scope of CSS downwards over time, by migrating functions to other system components, as they appear, and continue to improve interworking with these components. This will enable continued and improved integration across systems.

The technical issues are being managed by a number of activities which include:

- purchase of Hitachi Skyline processors to manage the larger systems,
- activities to reduce the load and improve processing efficiency,
- exploitation of the new versions of IDMS.

The major recommended migration steps are:

- continue and accelerate the deployment of client/server and the CSS re-engineering activities,
- invest in approaches to systems interworking, particularly between CSS and emerging provisioning systems, e.g. COSMOSS, to provide a more flexible order-handling platform and job-management platform,
- develop provisioning capability for volume products, either by use of existing, more modern, flexible systems and/or building new mid-tier applications,

- migrate customer information and associated function to the operational customer database when deployed,
- invest in CSS billing to improve flexibility and robustness to ensure it can both meet the company's need for a volume consumer-billing system and start the migration to new technology,
- move business billing to MSBS as fast as is practicable,
- move line-test function to mid-tier applications with the longer term aim of moving all problem management off CSS to a new mid-tier problem management system,
- continue current investigations on decoupling sub-systems, with the aim of proving these migration techniques, e.g. RADAR, NJR,
- continue to investigate migration options and technologies to identify industry best practice, and hence further techniques, for evolving CSS, such as transition engineering.

In the long term CSS must be replaced by more flexible platforms that align with current and future business needs. Addressing the issue from the point of view of the technical platform alone does not solve the fundamental problems, if CSS was re-implemented on an Oracle or DB2 database in its current form the problems would still exist. To realise the business benefits a major rewrite of CSS would be essential.

The optimum way forward is gradual replacement via the development of alternative platforms with which CSS interworks and which can gradually take over CSS functions. It is critical to the migration of CSS and BT's ability to meet on-going business requirements that the alternative architectures and systems are developed speedily within a clear overall framework.

Acknowledgements

Many people have contributed to the development of CSS over many years and have been involved in debates about its future. The key people who have contributed to the work described in this paper are Phil Dance, Jenny Keech, Robin Putney, Jeanne Keir, Dave Haxell, Jim Peters, Simon Whaley, Graham Woodman and Cathy Bland. Thanks are also due to the many people who contributed to the 'Essential Guide to CSS' [5], on which section 2 of this paper was heavily based.

References

- 1 Harrison P and Keech J: 'CSS design evolution feasibility study: proposed scope of CSS', Issue 1.0, Internal BT report (1994).
- 2 CSS Design Team: 'CSS design evolution feasibility study: design options', Issue 1.0, Internal BT report (1994).
- 3 Harrison P F: 'CSS evolution strategy 1996', Issue 1.0, Internal BT report (August 1996).
- 4 Vanstone S: 'CSS programme plan', Internal BT document (1966).
- 5 Brody C and Noble F (Eds): 'The essential guide to CSS', Issue 2, Internal BT document (September 1995).
- 6 Calladine J: 'BT Middleware — software as infrastructure', BT Technol J, 15, No 1, pp 135—146 (January 1997).
- 7 Selley C J: 'SMART — improving customer service', BT Technol J, 15, No 1, pp 69—80 (January 1997).
- 8 Whittaker S J et al: 'Improving customer service through speech technology', BT Technol J, 14, No 2 pp 75—83 (April 1996).
- 9 White P: 'A service system for BT's major customers', BT Technol J, 15, No 1, pp 81—86 (January 1997).
- 10 Burd E, Munro M and Wezeman C: 'Analysing large Cobol programs: the extraction of reusable modules', Proceedings of IEEE International Conference for Software Maintenance (November 1996).
- 11 Heritage A: 'Reverse engineering of CSS billing code into an object oriented design: feasibility study', Internal BT report (December 1991).
- 12 Regan T, Wezeman C, Kerr B and Thompson P: 'VAT experiment in OO', Internal BT report (March 1994).
- 13 Clark T: 'Corporate software reuse programme: re-engineering of OHMOLDC (Phase 2)', Internal BT report (September 1994).
- 14 Harrison P and Clark T: 'Corporate software reuse programme: software component assessment process', Internal BT report (February 1994).
- 15 Hall G: 'CSS tools newsletter', Internal BT report (1996).
- 16 Evans M: 'Managing change within BT's operational support systems', BT Technol J, 15, No 1, pp 180—185 (January 1997).
- 17 Brodie M L and Stonebraker M: 'Migrating legacy systems', Morgan Kaufman (1995).
- 18 Foraker-Blackwell T: 'The legacy application dilemma: maintaining or rebuilding', Advanced Software Developments Select, New Science Associates (1994).

- 19 Mathieson G: 'Customer handling intermediate server — an architecture led project', BT Technol J, 15, No 1, pp 114—134 (January 1997).
- 20 Bonell J, Kennedy N and Lumpkin A: 'Provisioning strategy', Internal BT report (1996).
- 21 Maxwell C and Wade B: 'Problem management strategy', Internal BT report (1996).
- 22 Chandler J: 'COSMOSS — designing for a changing world', BT Technol J, 15, No 1, pp 87—97 (January 1997).



Paul Harrison joined BT in May 1986 after a career as a lecturer in mathematics and computing in further and higher education. He graduated from the Polytechnic of Central London in 1974 with a BSc in Mathematics and Computing and later gained an MSc in Computer Science at City University.

He joined BT to work in Computing Training and has since been involved in work on expert systems, development methods and tools, rapid application development, and reuse and reverse engineering. He is currently the CSS Design Manager and leads a team carrying out design evolution activities as well as solution design for new products and services.